

3.Java servlets and XML

By Pradnya Lodha

Outline

Introduction to Server Side technology and TOMCAT,

Servlet: Introduction to Servlet, need and advantages, Servlet Lifecycle, Creating and testing of sample Servlet, session management.

JSP: Introduction to JSP, advantages of JSP over Servlet , elements of JSP page: directives, comments, scripting elements, actions and templates, JDBC Connectivity with JSP.

Servlet

- Introduction to server side Programming –
- Server side technology deals with the data submitted by the client
- Server side programming also act as a link between backend database and front end client side programming.
- Server side program basically handles the request made by the client. This program then prepares the response for the client.
- Various server side technologies include- servlet,JSP,ASP, .net , php and so on

Servlet

```
graph TD; Servlet --> SSSL[Server Side Scripting Language]; Servlet --> Dynamic[Dynamic]; Servlet --> DO[Database Operations are possible];
```

Server Side
Scripting
Language

Dynamic

Database
Operations
are possible

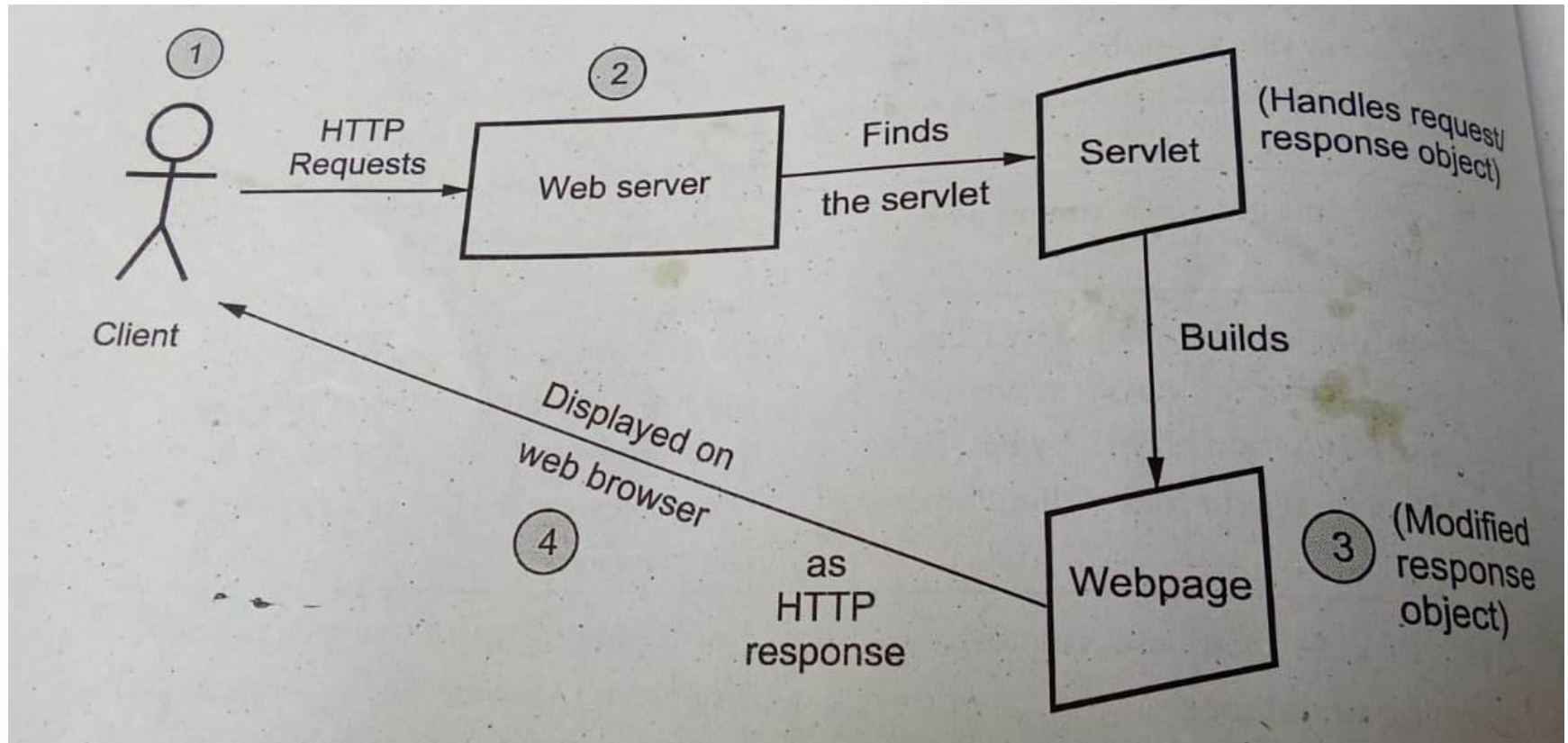
Difference between client side and server side scripting

Client-side scripting	Server-side scripting
Source code is visible to the user.	Source code is not visible to the user because its output of server-side is an HTML page.
Its main function is to provide the requested output to the end user.	Its primary function is to manipulate and provide access to the respective database as per the request.
It usually depends on the browser and its version.	In this any server-side technology can be used and it does not depend on the client.
It runs on the user's computer.	It runs on the webserver.
There are many advantages linked with this like faster response times, a more interactive application.	The primary advantage is its ability to highly customize, response requirements, access rights based on user.
It does not provide security for data.	It provides more security for data.
It is a technique used in web development in which scripts run on the client's browser.	It is a technique that uses scripts on the webserver to produce a response that is customized for each client's request.
HTML, CSS, and javascript are used.	PHP, Python, Java, Ruby are used.

Server Architecture Overview

- Servlets are basically the java programs that run on server.
- These are the programs that are requested by XHTML documents and are displayed in the browser window as a response to the request.
- The execution of servlet is managed by servlet container such as tomcat
- The servlet container is used in java for dynamically generate the web page on the server side.
- Servlet container is the part of web server that interacts with the servlet for handling the dynamic web pages from the client
- The main purpose of servlet is to add up the functionality to a web server.

Working of how servlet works?



Working of how servlet works?

- When a client make a request for some servlet, he/she actually uses the web browser in which request is written as a URL.
- The web browser then sends this request to web server. The web server first finds the requested servlet
- The obtained servlet gathers the relevant information in order to satisfy the client's request and builds a web page accordingly.
- This web page is then displayed to the client. Thus the request made by the client gets satisfied by the servlets

Need and Advantages

- Servlets can process and store the data submitted by the HTML form.
- Servlets are useful for providing the dynamic contents. For ex. Retrieving and updating the databases.
- Servlets can be used in the cookies. Cookies are small programs which can make use of the information submitted on currently accessed web pages.
- Servlets are used in session tracking. The session tracking are the useful programs that keeps track of all previously accessed the web pages

Advantages

- The servlet are very efficient in their performance
- The servlet are platform independent and can be executed on different web servers.
- The servlets working is based on request -response
- Servlet provide a way to generate the dynamic document.
- Using servlets multiple requests can be synchronized and then be concurrently handled.
-

Disadvantage

- . Servlet is a mixture of java skills and web related HTML skills.hence it is difficult to write servlets.
- . It slows down application execution
- . The java runtime environment is required on the server , if you want to execute servlet application.

A “Hello World” Servlet

- When you write a servlet program , it is necessary to i) Either implement servlet interface or ii) Extend a class that implements servlet interface
- When implementing servlet interface we must include `javax.servlet` package.hence the first line in our servlet program must be
- `import javax.servlet.*;`
- `GenericServlet` class is predefined implementation of servlet interface . Hence we can extend `GenericServlet` class in our servlet program.
- Similarly , the `HTTPServlet` class is a child class of `GenericServlet` class, hence we can extend this class as well while writing the servlet program.
-

Hence following are the two ways by which we can write servlet program

- `import javax.servlet.http.*;`
- `import javax.servlet.*;`
- `import java.io.*;`
- `public class DemoServlet extends HttpServlet`
- `{`
- `// body of servlet`
- `}`

- `import javax.servlet.http.*;`
- `import javax.servlet.*;`
- `import java.io.*;`
- `public class DemoServlet extends
GenericServlet`
- `{`
- `// body of servlet`
- `}`

- The servlet gets request from the client for some device. the servlet then processes the request and send the response back to the client . In order to handle these issues `HttpServletRequest` and `HttpServletResponse` are used in servlet program
- Thses methods are handled with the help of some methods that are popularly known as methods of `HttpServlet`

HttpServlet Methods

- This method is used to handle the GET request on the server-side.
- The GET type request is usually used to preprocess a request.
- The doGet method requests the data from the source.

Syntax:

- protected void doGet(HttpServletRequest request, HttpServletResponse response)
- throws ServletException, IOException
- Parameters:
 - request – an HttpServletRequest object that contains the request the client has made of the servlet.
 - response – an HttpServletResponse object that contains the response the servlet sends to the client.
- Exceptions:
 - IOException – if an input or output error is detected when the servlet handles the GET request.
 - ServletException – Use to handle the GET request.

doPost() Method

- This method is used to handle the POST request on the server-side.
- This method allows the client to send data of unlimited length to the webserver at a time.
- The doPost method submits the processed data to the source

Syntax:

- protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
- Parameter:
- request – an HttpServletRequest object that contains the request the client has made of the servlet.
- response – an HttpServletResponse object that contains the response the servlet sends to the client.
- Throws:
- IOException – if an input or output error is detected when the servlet handles the POST request.
- ServletException – Use to handle the POST request.

doPut() Method

- This method is overridden to handle the PUT request.
- This method allows the client to store the information on the server(to save the image file on the server).
- This method is called by the server (via the service method) to handle a PUT request.

Syntax:

- `protected void doPut(HttpServletRequest request, HttpServletResponse response)`
- throws `ServletException`, `IOException`
- Parameters:
 - `request` – an `HttpServletRequest` object that contains the request the client has made of the servlet.
 - `response` – an `HttpServletResponse` object that contains the response the servlet sends to the client.
- Throws:
 - `IOException` – if an input or output error is detected when the servlet handles the PUT request.
 - `ServletException` – Use to handle the PUT request.

doDelete() Method

- This method is overridden to handle the DELETE request.
- This method allows a client to remove a document or Web page from the server.
- While using this method, it may be useful to save a copy of the affected URL in temporary storage to avoid data loss.

Syntax:

- `protected void doDelete(HttpServletRequest request, HttpServletResponse response)` throws `ServletException`, `IOException`
- Parameters:
 - `request` – an `HttpServletRequest` object that contains the request the client has made of the servlet.
 - `response` – an `HttpServletResponse` object that contains the response the servlet sends to the client.
- Exceptions:
 - `IOException` – if an input or output error is detected when the servlet handles the `DELETE` request.
 - `ServletException` – Use to handle the `DELETE` request.

Servlet Life Cycle

- In the life cycle there are three important methods. These methods are -
 - 1. Init
 - 2. Service
 - 3. Destroy
- The client enters the URL in the web browser and makes a request. The browser then generates the HTTP request and sends it to the web server.

Init () method

- The init method is called only once. It is called only when the servlet is loaded in memory for the first time.
- it is used for one-time initializations.
- When a user invokes a servlet, a single instance of each servlet gets
- created, with each user request resulting in a new thread that is
- handed off to doGet or doPost as appropriate.
- The init() method simply creates or loads some data that will be used throughout the life of the servlet.
- The init method definition looks like this –
- `public void init() throws ServletException {`
- `// Initialization code...`
- `}`

The service() Method

- The service() method is the main method to perform the actual task. The servlet container (i.e. web server) calls the service() method to handle requests coming from the client(browsers) and to write the formatted response back to the client.
- When server receives a request for a servlet, the service()
- method checks the HTTP request type (GET, POST) and calls
- doGet, doPost, methods as appropriate.
- Here is the signature of this method –
- `public void service(ServletRequest request, ServletResponse`
- `response) throws ServletException, IOException { }`

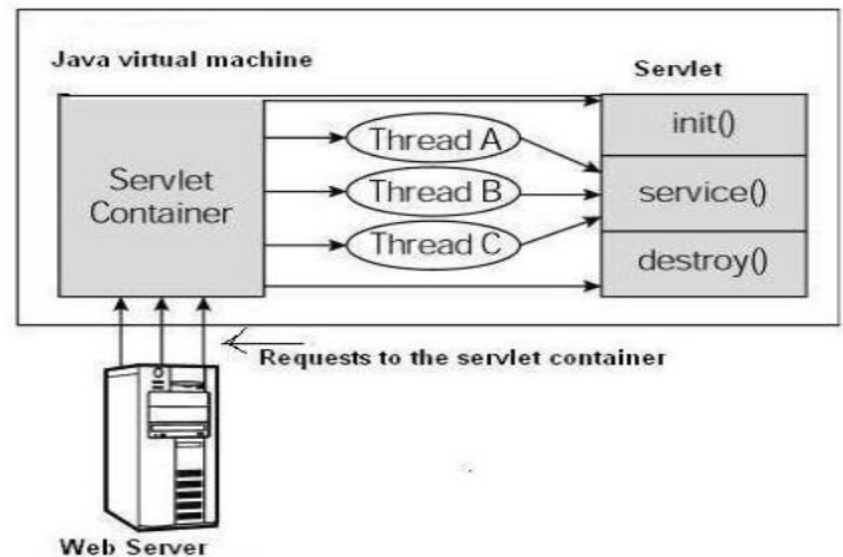
The destroy() Method

- The destroy() method is called only once at the end of the life cycle of a servlet.
- This method gives your servlet a chance to close database connections, halt background threads, and perform other such cleanup activities.
- After the destroy() method is called, the servlet object is marked for garbage collection.
- The destroy method definition looks like this –
- `public void destroy() {`
- `// Finalization code...`
- `}`

Architecture Diagram

In servlet for one session if multiple request then a single process is created and inside the process different threads are created for each request. All threads call to service method. servlet method initialize init method. When servlet need to destroy it call the destroy method.

- First the HTTP requests coming to the server are delegated to the servlet container.
- The servlet container loads the servlet before invoking the service() method.
- Then the servlet container handles multiple requests by spawning multiple threads, each thread executing the service() method of a single instance of the servlet.



How to write servlet program

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*; // Extend HttpServlet class
public class FirstServlet extends HttpServlet
{
} public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException
Response.setContentType("text/html")
PrintWriter out = response.getWriter();
out.println("<h1>" Hello "</h1>");
}
}
```

Program Explanation

- `import java.io.*;` package is used for taking care of I/O operation
- `javax.servlet.*` and `javax.servlet.http.*` are important packages containing the classes and interfaces that are required for operation of servlet.
- `javax.servlet.http` package `HttpServletRequest` and `HttpServletResponse` are two commonly used interfaces.
- We had given class name `FirstServlet` which should be derived from the class `HttpServlet`
- Then we have defined `doGet` method to which the HTTP request and response are passed as parameter
- A MIME type (also known as a Multipurpose Internet Mail Extension) is a standard that indicates the format of a file. MIME type is specified as using the `setContentType()` method. In this method "text/html" is specified as the MIME type.
- Output stream is created using `PrintWriter()`. The `getWriter()` method is used for obtaining the output stream

Server generating dynamic program

- The web site must be able to generate dynamic contents as per requirements. servlet can generate static, dynamic or interactive web pages
- Following is a simple servlet program that displays the current date and time. Note that time is continuously changing and on refreshing the same page.

Example

```
import java.io.*;
import javax.servlet.*;

public class dynamicServletdemo extends HttpServlet
{
}

public void services(HttpServletRequest request, HttpServletResponse response)
    throws IOException, ServletException
{
    Response.setContentType("text/html")
    PrintWriter out = response.getWriter();
    java.util.Date date = new java.util.Date ();
    out.println("<h1>"+ " "current date and time" +date.toString()+ "</h1>");
    Out.close();
}
}
```

// we have used Date() method of java.util class that is intended to display current date and time.

The date is displayed using the date.toString() method

Parameter data

- Many times we need to pass some information from web browser to web server.
- In such case the html document containing the FORM is written which sends the data to the servlet present on the web server.
- The browser uses two methods to pass this information to web server. These methods are GET method and POST method
- To make the form works with Java servlet, we need to specify the following attributes for the <form> tag:
 - 1) method = "method name" : to send the form data as an HTTP POST request to the server.
 - 2) action -= "URL/address of the servlet" : specifies realtive URL of the servlet which is responsible for handling data posted from this form.

How does servlet read from data ?

- Servlet makes use of following three methods to read the data entered by the user on the HTML form
- 1) `getParameter()` - you call `request.getParameter()` method to get the value of a form parameter.
- 2) `getParameterValues()` – Call this method if the parameter appears more than once and returns multiple values, for example checkbox.
- 3) `getParameterNames()` – Call this method if you want a complete list of all parameters in the current request.

Example : To read data from HTML file and print that .

- It requires 2 files:
 1. HTML (s1.html)File
 2. Servlet (s2.java) File
- First Run HTML file and after clicking on submit button it will run servel(.java) file.

Example

- We will write two source files first one is the HTML file names test.html in which the list of all the desired languages is displayed along with the submit button.
- This data made by this request will be received by the servlet named my_choiceservlet.java which reads the selected parameter and displays the message about the selection made

Test.html

```
<html>
<body>
<form name = "form1" method = GET
    action = "http ://localhost:4040/servlets - examples/servlet/my_choiceservlet">
<b>language :</b>
<select name = "language" size = "1">
<option value = "c">c</option>
<option value = "c++">c++</option>
<option value = "java"> java </option>
<option vale = "c#">c#</option>
</select>
<br> <br>
<input type = "submit" value = "submit">
</form>
</body>
</html>
```

My_choiceservlet.java

```
•      import javax.servlet.http.*;

•      import javax.servlet.*;

•      import java.io.*;

•      Public class my_choiceservlet extends HttpServlet

•      {

•      public void doGet(HttpServletRequest request, HttpServletResponse response)

•      throws ServletException,IOException

•      {

•      String lang = req.getParameter("Language");

•      Res.setContentType("text/html");

•      PrintWriter out = res.getWriter();

•      Out.println ("The selected language is ~ "+lang);

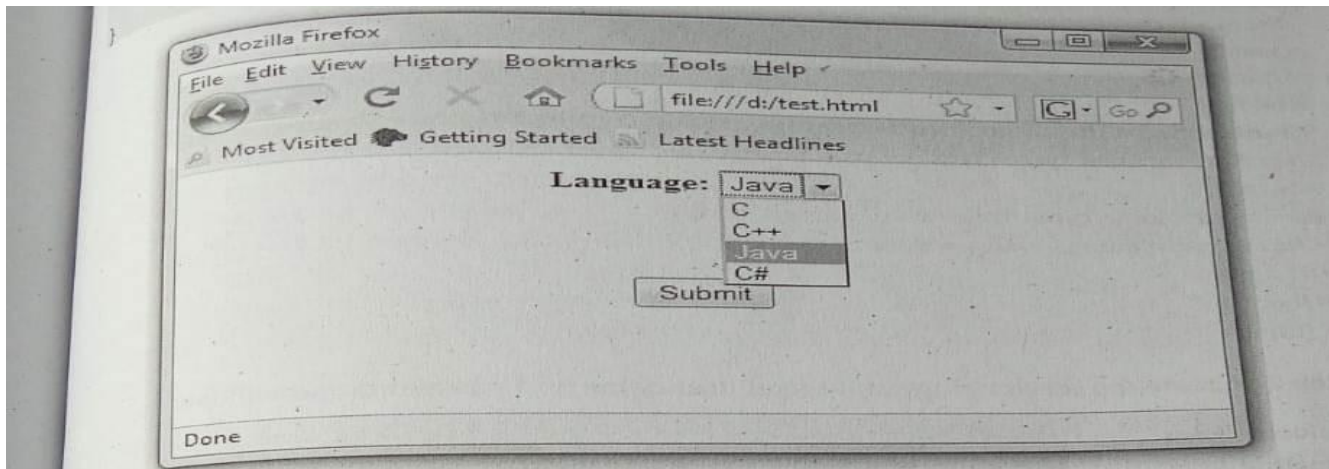
•      Out.close();

•      }

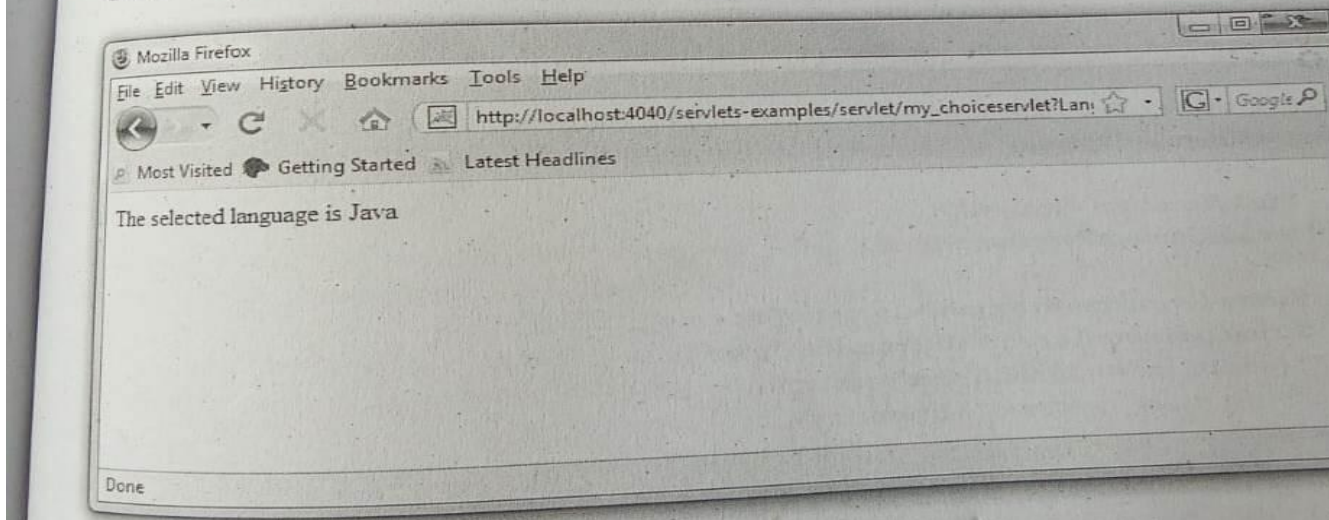
•      }

•
```

Output



On clicking the submit button we will get following output.



session

- When you open some application, use it for some time and then close it. This entire scenario is named as session.
- Sometimes the information about the session is required by the server. This information can be collected during the session. This process is called session tracking.
- There are three commonly used techniques used for session tracking and those are -
 - Session id
 - Cookies
 - URL rewriting

Session management using Session ID

- Session Tracking is a way to maintain state (data) of an user. It is also known as session management in servlet.
- Http protocol is a stateless so we need to maintain state using session tracking techniques.
- Each time user requests to the server, server treats the request as the new request.
- So we need to maintain the state of an user to recognize to particular user.
- For sending all state information to and from between browser and server usually id is used . This ID is basically a session-ID

Session management using Session ID

- Thus session ID is passed between the browser and server while processing the information. This method is keeping track of all the information between the server and browser using **sessionID** is called session tracking.
- Why use Session Tracking?
 - To recognize the user It is used to recognize the particular user.

Session Tracking Techniques

- cookies - cookies are some little information that can be left on your computer by the other computer when we access the internet.
- Generally this information is left on your computer by some advertising agencies on the internet. Using the information stored in the cookies these advertising agencies can keep track of your internet usage liking of users
- For the application like on-line purchase system once you enter your personal information such as your name or your email-id then it can be remembered by these system with the help of cookies.
- Sometimes cookies are very much dangerous because by using information from your local disk some malicious data may be passed to you. So it is up to you how to maintain your own privacy and security.

Types of Cookie

There are 2 types of cookies in servlets.

- Non-persistent cookie
 - Persistent cookie
-
- Non-persistent cookie - It is valid for single session only. It is removed each time when user closes the browser.
 - Persistent cookie - It is valid for multiple session . It is not removed each time when user closes the browser. It is removed only if user logout or signout.

Cookie class

javax.servlet.http.Cookie class provides the functionality of using cookies. It provides a lot of useful methods for cookies.

Constructor of Cookie class

Constructor	Description
Cookie()	constructs a cookie.
Cookie(String name, String value)	constructs a cookie with a specified name and value.

Useful Methods of Cookie class

There are given some commonly used methods of the Cookie class.

Method	Description	
public void setMaxAge(int expiry)	Sets the maximum age of the cookie in seconds.	
public String getName()	Returns the name of the cookie. The name cannot be changed after creation.	
public String getValue()	Returns the value of the cookie.	
public void setName(String name)	changes the name of the cookie.	
public void setValue(String value)		changes the value of the cookie.

Creation of cookies

- The cookie can be created in servlet and added to response object using addCookie method.

```
Cookie ck=new Cookie("user","sonoo jaiswal");//creating cookie object
```

```
response.addCookie(ck);//adding cookie in the response
```

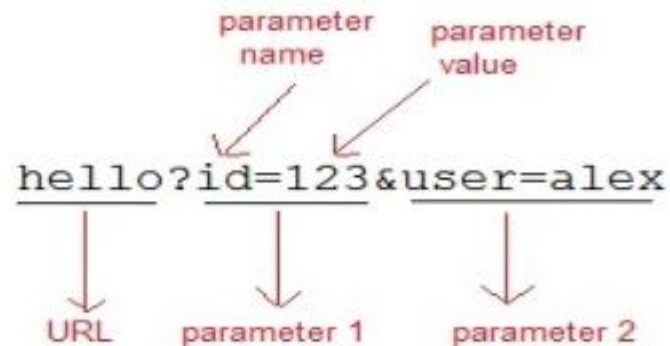
Reading cookies

The value from the cookie can be obtained using getName() and getValue() methods.

```
Cookie ck[]=request.getCookies();  
for(int i=0;i<ck.length;i++)  
{  
    out.print("<br>" + ck[i].getName() + " " + ck[i].getValue());  
    //printing name and value of cookie  
}
```

URL Rewriting

- If the client has disabled cookies in the browser then session management using cookie wont work.
- In that case **URL Rewriting** can be used as a backup. **URL rewriting** will always work.
- In URL rewriting, a token(parameter) is added at the end of the URL.
- The token consist of name/value pair separated by an equal(=) sign.
- **For Example:**



Session Tracking- URL Rewriting

- When the User clicks on the URL having parameters, the request goes to the **Web Container** with extra bit of information at the end of URL.
- The **Web Container** will fetch the extra part of the requested URL and use it for session management.
- The `getParameter()` method is used to get the parameter value at the server side.
- **Advantage of URL Rewriting**
 - It will always work whether cookie is disabled or not (browser independent).
 - Extra form submission is not required on each pages.
- **Disadvantage of URL Rewriting**
 - It will work only with links.
 - It can send only textual information.

Other servlet capabilities

In this section we will discuss various methods of `HttpServletRequest` and `HttpServletResponse` class

Additional HttpServletRequest Methods –

- String [`getRemoteAddr\(\)`](#) - Returns the IP address of the client who is making request
- String `getRemoteHost()` - it tries to retrieve the host name. If host name is empty, then it tries to retrieve the IP address, just like how `getRemoteAddr()` works
- String `getProtocol ()` – this method returns the name of the protocol
- String `getHeader(String field)` – when the field name is given the value for the header is returned
- StringBuffer `getRequestURL()` – this method returns the string containing the URL which is used to access the servlet.

Additional HttpServletRequest Methods

- `Void addCookie(Cookie cookie)` – Adds the specified cookie to the response
- `Void sendRedirect(string location)` – sends a temporary redirect response to the client using the specified redirect location URL and clears the buffer.
- `int getStatus ()` – gets the current status code of this response.
- `String getHeader (String,name)` – gets the value of the response header with the given name
- `void setHeader (String name, String value)` – sets a response header with the given name and value
- `void setStatus (int cs)` – sets the status code for this response.
- `void sendError (int sc, String msg)` – sends an error response to the client using the specified status and clears the buffer.

Data Storage

- Typical web application contains the HTML document that often interact with some form of data storage.
- This data storage can be a file system or database management system.
- The efficient use of file system or database management system is required by every web application.
- There are two commonly used techniques for data storage – java Object Serialization and Java Database Connectivity (JDBC).
- Serialization in Java is a mechanism of writing the state of an object into a byte – stream.
- JDBC is an API used to assist the java programs to interact with the database management systems (popularly handled using MySQL)

- Serialization is the process of writing the state of the object to the byte stream.

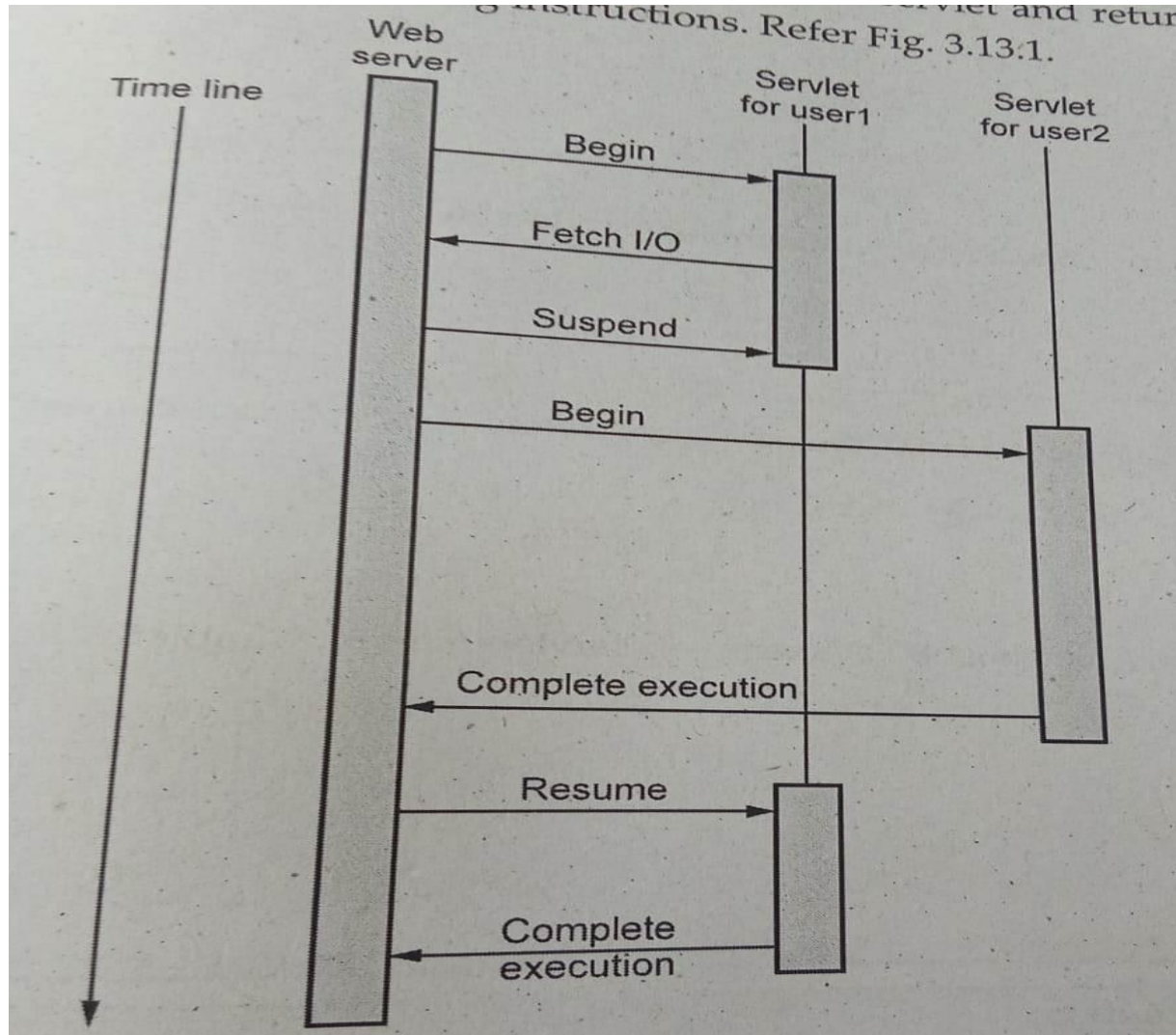
This technique is useful when we want to store the current state of the object to the file.

- For serialising the object we need to implement serializable interface. This interface does not define any member.

Servlet: Concurracy

- concurrency allows multiple userd to access the system. It also allows system program to execute concurrently
- for instance , when web browser is loading some image, at the same time it also responds to mouse clicks. This type of concurrency is achieved at the client end in clients browser.
- On the server side the situation is different. There can be hundreds or even thousands of users simultaneously accessing a web site,
- Concurrent web servers handle multiple users as follows – it begin with one servlet , executes some instruction , then suspends it for a while and start executing the other servlet.
- Eventually completes the second servlet and returns back to first servlet to execute the remaining instructions.

Concurrency achieved by web browser



Concept of Thread

- Thread is a light weight process. It is used for concurrent execution of the tasks.
- The server creates a thread for each HTTP request.
- The data structure maintains the information about the state of the thread along with the references to the `HttpServletRequest` and `HttpServletResponse` object created by the server.
- The java virtual machine also maintains the information whether the particular thread is currently running or not. when one thread is suspended then the state of other executing thread is loaded into java virtual machine.

Database (MySql)and Java Servlet

- JDBC is a Java API for database access. A servlet can connect to MYSQL and sends the SQL commands to the database and get the required data.
- Hence we will first get introduced with structured query language SQL.
- Structure query language using MySql – the MySQL is a open source database product which can be downloaded from the web site
- After getting installed on the machine the command prompt window for MYSQL can appear

SQL query statement

1 . creating database -

```
mysql> create database demoprj;
```

Query OK, 1 row affected (4.10 sec)

//on duplicate key statements, the affected-rows value per row is 1 if the row is inserted as a new row, 2 if an existing row is updated, and 0 if an existing row is set to its current values.

- Displaying all the database –

```
mysql>SHOW DATABASES
```

- Creating table – we must create a table inside a database hence it is a common practice to use create table command after use database command.

While creating a table we must specify the table fields.

```
Mysql> CREATE TABLE my_table(id INT(4),name VARCHAR(20));
```

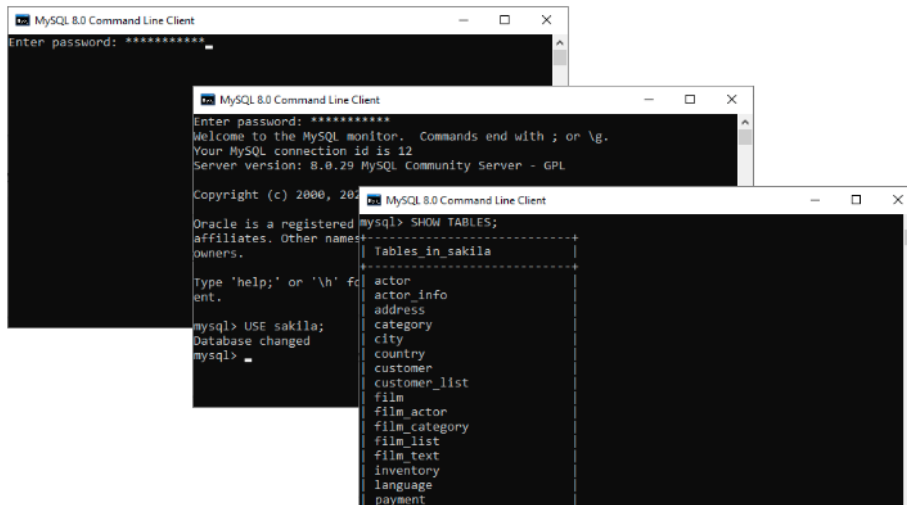
```
Query OK, 0 rows affected(0.28 sec)
```

- Use of primary key - the primary key contains the unique value.the primary key column can not contain NULL value.
- Each table can have only one primary key.following is a SQL statement used to create PRIMARY KEY.

```
CREATE TABLE student_table(  
  
roll_no INT (4) NOT NULL AUTO_INCREMENT,  
  
Name VARCHAR(50) NOT NULL,  
  
PRIMARY KEY (roll_no)  
  
};
```

Displaying table – after creating the table using SHOW command we can see all the existing tables in the current database

mysql> SHOW TABLES



The image shows three overlapping screenshots of the MySQL 8.0 Command Line Client window. The top window shows the password prompt. The middle window shows the login success message and the prompt. The bottom window shows the execution of the SHOW TABLES command and the resulting list of tables in the sakila database.

```
mysql> SHOW TABLES;
```

Tables in sakila
actor
actor_info
address
category
city
country
customer
customer_list
film
film_actor
film_category
film_list
film_text
inventory
language
payment

MySQL SHOW TABLES command example

To use the SHOW TABLES command, you need to log on to the MySQL server first.

1. On opening the MySQL Command Line Client, enter your password.
2. Select the specific database.
3. Run the SHOW TABLES command to see all the tables in the database that has been selected.

- Try - <https://www.javatpoint.com/mysql-show-list-databases>

- Inserting values into the table -we can insert only one complete record at a time. It is as shown below
- Mysql > INSERT INTO my_table
- → VALUES (1,'SHILPA');
- Query OK , 1 row affected (0.05 sec)
- Displaying the contents of the table –
mysql> SELECT *FROM my_table

- We can also write SELECT statement for selecting particular row by specifying some condition such as –

```
Mysql > SELECT *FROM my_table where id = 1;
```

Or

```
Mysql> SELECT *FROM my_table where name = 'SHILPA';
```

- Thus we can insert the rows into the table by repeatedly giving the INSERT command.
- If we want to get the records in sorted manner then we use ORDER BY clause

example

```
mysql > SELECT * FROM my_table;
```

```
Mysql> SELECT *FROM my_table ORDER BY name
```

- Updating the record – for updating the record in the database following command can be used –
Mysql > UPDATE my_table
-> SET name = 'PRIYANKA'
-> WHERE id = 4;
- Deleting record – for deleting particular record from a database
- Mysql > DELETE FROM my_table
- ->where id = 3
- For deleting the table
- the table can be deleted using the command
- mysql>drop table my_table

Inner join - The INNER JOIN keyword selects records that have matching values in both tables.

Syntax -

```
SELECT column_name(s)
```

```
FROM table1
```

```
INNER JOIN table2
```

```
ON table1.column_name = table2.column_name;
```

Order by clause -

If we want to get the records in sorted manner then we use ORDER BY clause.

```
Mysql > SELECT * FROM my_table
```

```
Mysql> SELECT *FROM my_table ORDER BY name;
```

JDBC perspective

- JDBC stands for JAVA Database Connectivity
- JDBC is nothing but API(Application Programming Interface)
- It consists of various classes, interfaces exception using java application can send SQL statements to a database
- JDBC is specially used for having connectivity with RDBMS packages (such as ORACLE or MYSQL) using corresponding JDBC driver.
- This JDBC API is defined in the java.sql and javax.sql packages
- We use following core JDBC classes and interfaces that belong to java.sql package
- Driver manager – when java application needs connection to the database it invokes the DriverManager class. This class then loads JDBC driver in the memory
- The driver manager also attempts to open a connection with the desired database.
- For JSP -MYSQL connectivity we can get DriverManager class as -
- `Class.forName("sun.jdbc.odbc.jdbcodbcDriver");`

1. connection – this is an interface which represents connectivity with the data source.

- The connection is used for creating the statement instance. After loading the driver, establish connections as shown below as follows:

```
Connection con = DriverManager.getConnection(url,user,password)
```

- user: Username from which your SQL command prompt can be accessed.
- password: password from which the SQL command prompt can be accessed.
- con: It is a reference to the Connection interface.
- Url: Uniform Resource Locator which is created as shown below:

```
String url = “ jdbc:oracle:thin:@localhost:1521:xe”
```

2 . statement – this interface is used for representing the sql statements some example of SQL STATEMENTS are -

```
SELECT * FROM my_table
```

```
UPDATE students_table set name = “Neel” Where roll_no ='1';
```

Use of JDBC Statement is as follows:

How JDBC works?

1) **SQLException** - For handling SQL exceptions

3.14.2.1 How JDBC Works ?

Following is a way by which JDBC works -

- 1) First of all Java application establishes connection with the data source.
- 2) Then Java application invokes classes and interfaces from JDBC driver for sending queries to the data source.
- 3) The JDBC driver connects to corresponding database and retrieves the result.
- 4) These results are based on SQL statements which are then returned to Java application.
- 5) Java application then uses the retrieved information for further processing.

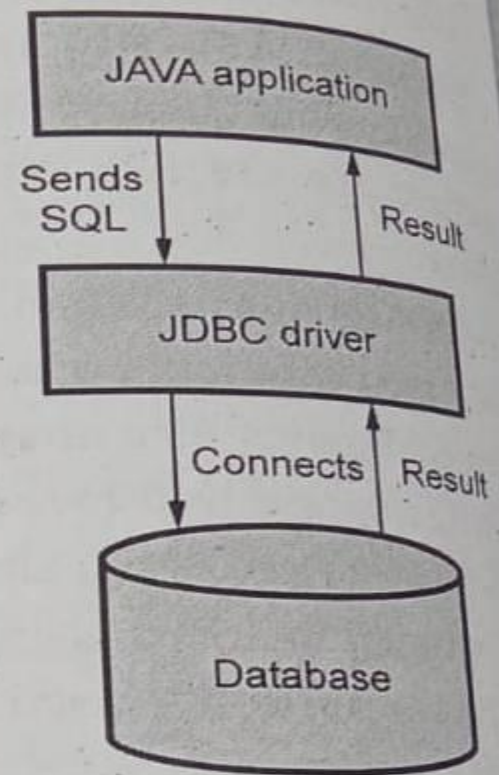


Fig. 3.14.1 Role of JDBC

Difference between JDBC and ODBC

- The JDBC is similar to the ODBC that is open database connectivity . But the ODBC is used for managing any kind of database and JDBC is specially created to support the java programs.
- Uses of JDBC -
 1. the JDBC helps the client to store and retrieve the data to the databases.
 - 2.JDBC allows the client to update the databases

Steps for connectivity between Servlet and Database

Following steps are used to connect JDBC to MYSQL

- Step 1 : To access and process the Database operations, we need to import all the required “*java.sql*” packages in the program.

```
import java.sql.*
```

- Step 2 :Load JDBC Driver.

The JDBC driver for MYSQL can be loaded using following statement

We can register the driver by using “*Class.forName()*”

```
method:Class.forName("org.postgresql.Driver"); //// Register
```

PostgreSQL Driver

Now, we need to establish the connection to the database using the "DriverManager.getConnection()" method.

```
String URL = "jdbc:postgresql://localhost:5432/";
```

```
String USER = "postgres";
```

```
String PASSWORD = "postgres";
```

```
Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);
```

In this example, we need to pass the URL, Username, and Password of the database we are using as parameters to the connection.

URL: This is the address of the database you are using. In URL, you need to mention the "jdbc:postgresql://localhost:5432/".

Username and Password: You need to specify the username and password to your database.

➤ Step 4: Create Statement

After successful connection, prepare JDBC statement object using the Connection object.

```
Statement stmt = conn.createStatement();
```

➤ Step 5: Execute Query

Construct the SQL query based on the client request and execute the query using statement object.

```
stmt.executeQuery(sql);
```

Step 6:

Display the result.

XML

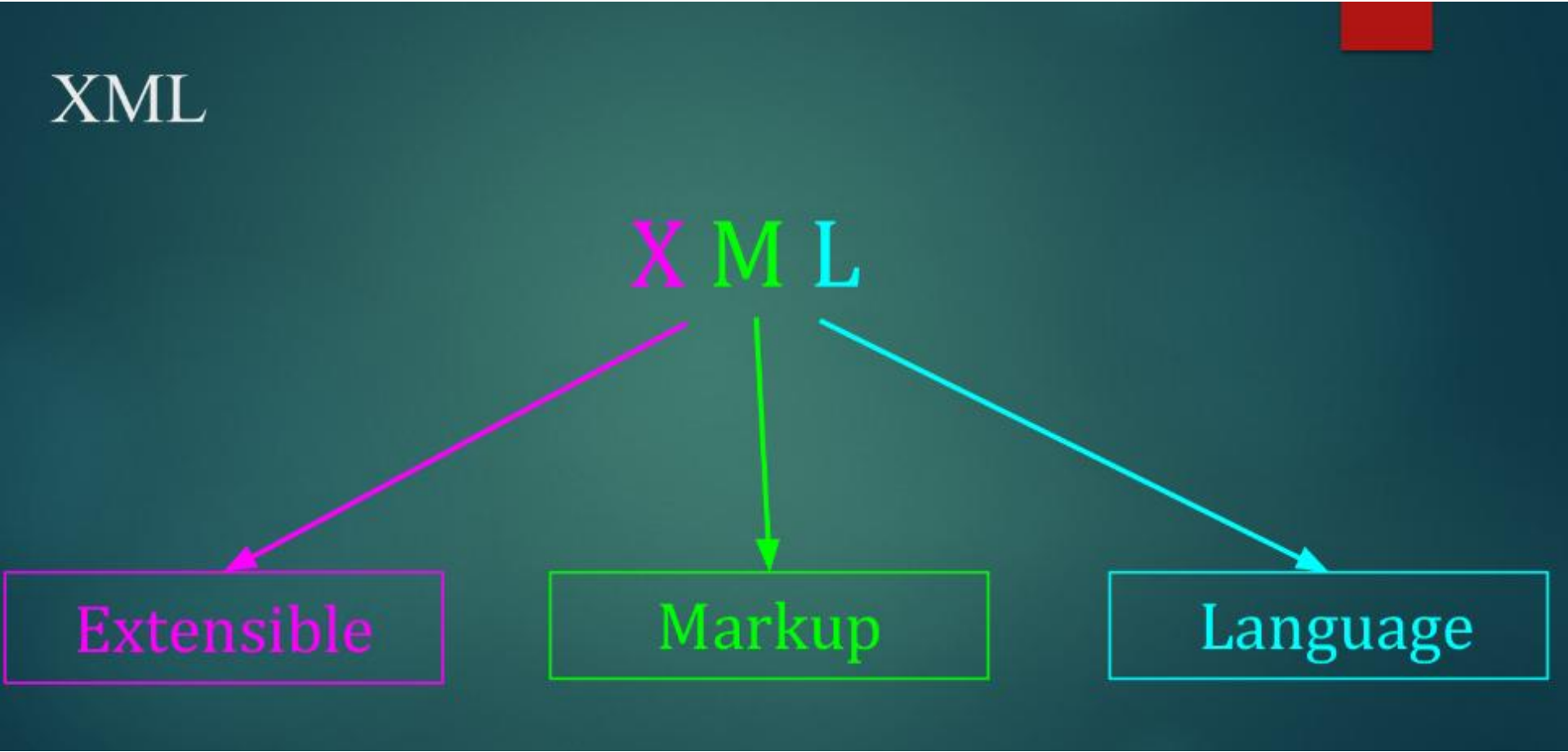
XML

XML

Extensible

Markup

Language



What is XML?

- XML stands for eXtensible Markup Language
- XML is a markup language much like HTML
- XML was designed to store and transport data
- XML was designed to be self-descriptive
- XML is a W3C Recommendation

Advantages of XML

- XML is platform independent and programming language independent, thus it can be used on any system and supports the technology change when that happens.
- The data stored and transported using XML can be changed at any point of time without affecting the data presentation
- Every XML document has a tree structure. Hence complex data can be arranged systematically and can be understood in simple manner.
- The XML document is language neutral. That means a java program can generate an XML document and this document can be parsed by perl.

XML Uses

XML used to store & arrange Data

XML used to exchange the information between organizers and systems

XML use to simplify the creation of a HTML document

XML can easily be merged with style sheets to create desired output.

used for unloading and reloading a database

XML ican express any type of XML document

XML Vs HTML

XML	HTML
Is Case Sensitive	Is not Case Sensitive
Has user defined tags	Has its own predefined tags
Used for Transferring Data	Used for Displaying Data
Closing tags are mandatory	Closing tags are not always mandatory
It is Dynamic	It is Static
XML preserve white spaces	HTML does not preserve white spaces

XML Declaration

Elements and Attribute – Various building blocks of XML are –

1. Elements – the basic entity is element. The elements are used for defining the tags. the elements typically consist of opening and closing tag. Mostly one element is used to define a single tag.
2. Attribute - the attribute are generally used to specify the values of the element . These are specified within the double quotes.

Ex - <flag type = "True">

The type attribute of the element flag is having the value true.

3. CDATA – CDATA stands for character Data. This character data will be parsed by the parser
4. PCDATA – it stands for Parsed Character Data

Declaring Element

XML elements can be defined as building blocks of an XML. Elements can behave as containers to hold text, elements, attributes, media objects or all of these.

Each XML document contains one or more elements, the scope of which are either delimited by start and end tags, or for empty elements, by an empty-element tag.

Syntax

Following is the syntax to write an XML element –

`<element-name attribute1 attribute2> ....content </element-name>`where,

element-name is the name of the element. The *name* its case in the start and end tags must match.

attribute1, attribute2 are attributes of the element separated by white spaces. An attribute defines a property of the element. It associates a name with a value, which is a string of characters. An attribute is written as –

`name = "value"` *name* is followed by an = sign and a string *value* inside double(" ") or single(' ') quotes.

Declaring attribute

- The attribute type declaration is done using the string ATTLIST.
- The attribute type specifies the type of data which can be used for specifying the attribute
- The ATTLIST statement is used to list and declare each attribute that can belong to an element.

Syntax : <!ATTLIST elementName attributeName attributeType default >

Parameters

- elementName - The name of the element to which the attribute list applies.
- attributeName - The name of an attribute. This parameter can be repeated as many times as needed to list all attributes available for use with elementName.
- default - The default value for the attribute named in attributeName. The following table describes the possible defaults.

Following are some attribute type :

- The data type for the attribute named in the attributeName parameter, which must be one of the following:
- CDATA – The attribute will contain only character data.
- ID - The value of the attribute must be unique. It cannot be repeated in other elements or attributes used in the document.
- IDREF – The attribute references the value of another attribute in the document, of ID type.
- ENTITY – The attribute value must correspond to the name of an external unparsed ENTITY, which is also declared in the same DTD.
- ENTITIES - The attribute value contains multiple names of external unparsed entities declared in the DTD.
- NMTOKEN – The attribute value must be a name token. Name tokens allow character data values, but are more limited than CDATA. A name token can contain letters, numbers, and some punctuation symbols such as periods, dashes, underscores and colons. Name token values, however, cannot contain any spacing characters.
- NMTOKENS - The attribute value contains multiple name tokens. See the description for NMTOKEN and Enumerated for more information.
- Enumerated – The attribute values are limited to those within an enumerated list. Only values that match those listed are validly parsed. All enumerated data types are enclosed in a set of parentheses with each value separated by a vertical bar ("|").

Default - The default value for the attribute named in attributeName. The following table describes the possible defaults.

Defaults	Description
#REQUIRED	The attribute must appear in the XML document or a parsing error will result. To avoid a parse error in some cases, you can optionally use the <code>defaultValue</code> field directly following this keyword.
#IMPLIED	The attribute can appear in the XML document, but if omitted, no parsing error will result. Optionally, in some cases you can also use the <code>defaultValue</code> field directly following this keyword.
#FIXED	The attribute value is fixed in the DTD and cannot be changed or overridden in the XML document. If this keyword is used, the <code>defaultValue</code> field directly following this keyword must also be used to declare the fixed attribute value.
<i>defaultValue</i>	A default or fixed value. The parser inserts this value into the XML document when the attribute is missing or is not used in the XML document. All values must be enclosed in a set of quotation marks (either single or double quotes).

Example

- This example declares the following for the <book> element:
 - An optional publisher attribute that can only contain character data.
 - A fixed reseller attribute with its value is set to "MyStore".
 - A required ISBN attribute that must contain a unique identifying value for each <book> element in the XML document.
 - A required InPrint attribute that must contain either a "yes" or "no" value. The default enforces a "yes" value when the value is not explicitly set in the XML document.
 - XML
-
- <!ATTLIST book
 - publisher CDATA #IMPLIED
 - reseller CDATA #FIXED "MyStore"
 - ISBN ID #REQUIRED
 - inPrint (yes|no) "yes"
 - >

Declaring Entities

- The attribute type declaration is done using the string ENTITY.
- Inside the XML file at the beginning the entity can be defined using the <!DOCTYPE ...> and with the help of string ENTITY.
- ENTITYs are extremely useful for repeating information or large blocks of text that could be stored in separate files.

Rules

- Rules that must be followed while writing XML -
- 1. XML is a case sensitive. For example -
- `<Body>See Spot run.</body>`
- `<body>See Spot run.</body>`
- Is not allowed because the words Body and body are treated differently
- 2. in XML each start tag must have matching end tag. For example -
- `<body>See Spot run.</body>`
- `<body>See Spot catch the ball.</body>`
- 3. the elements in XML must be properly nested. For example
- `<b><i>This text is bold and italic.</b></i>` //Incorrect:
- `<b><i>This text is bold and italic.</i></b>` //Correct:

XML Namespaces

- Sometimes we need to create two different elements by the same name.
- The xml document allows us to create different elements which are having the common name. This technique is known as namespace.
- In some web documents it becomes necessary to have the same name for two different elements.
- Here different elements mean that elements which are intended for different purposes.

For example – consider the following xml document

```
<file-description>  
<text fname = "input.txt">  
<describe>it is a text file </describe>  
</text>  
<text fname = "flower.jpg">  
<describe> it is an image file</describe>  
</text>  
</file-description>
```

The above documents does not produce any error although the elements text is used for two different attribute values

Document type definition

- The document type definition is used to define the basic building block of any xml document.
- Using DTD we can specify the various elements types, attributes and there relationship with one another .
- Basically DTD is used to specify the set of rules for structuring data in any XML files
- For example – if we want to put some information about some students in XML file then, generally we use tag student followed by his/her name, address, standard and marks.
- That means we are actually specifying the manner by which the information should be arranged in the XML file. And for this purpose the document type definition is used.

Various building blocks of XML are -

1. Element – the basic entity is element.
 - The elements are used for defining the tags.
 - The elements typically consist of opening and closing tag.
 - Mostly only one element is used to define a single tag
2. Attribute – the attributes are generally used to specify the values for the element. These are specified within the double quotes.

for example - `<flag type = "true">`

The type attribute of element flag is having the value true.

3. CDATA - CDATA stands for character data. This character data will be parsed by the parser.
4. PCDATA – it stands for parsed character data (i.e. text). (XML parsing is the process of reading an XML document and providing an interface to the user application for accessing the document. An XML parser is a software apparatus that accomplishes such tasks.) Any parsable character data should not contain the markup characters.

The markup characters are `<` or `>` or `&`. If we want to use less than, greater than or ampersand characters then make use of `<`, `>`, or `&`

There are two ways by which DTD can be defined -

Internal DTD – Internal DTD is a type of Document Type Definition (DTD) in XML that is written within the XML document itself.

- It specifies the structure and rules for the elements and attributes of the XML document
- An internal DTD is enclosed within the <!DOCTYPE> declaration of the XML document and is defined using a set of predefined keywords and syntax.
- Internal DTDs are suitable for smaller XML documents where the complexity of the structure is not very high.
- It is easier to maintain and modify the internal DTD as it is part of the XML document itself.

example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE customers[ <!ELEMENT customers (customer+)>
  <!ELEMENT customer (name, email, phone)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT email (#PCDATA)>
  <!ELEMENT phone (#PCDATA)>
]>
<customers>
  <customer>
    <name>Satyam Nayak</name>
    <email>Satyam@Nayak.com</email>
    <phone>112-123-1234</phone>
  </customer>
  <customer>
    <name>Sonu N</name>
    <email>Sonu@N.com</email>
    <phone>112-455-9969</phone>
  </customer>
</customers>
```

here we are defining the
DTD internally

External DTD

- External DTD is a type of Document Type Definition (DTD) in XML that is located outside of the actual XML document it describes.
- It can be stored in a separate file or accessed via a URL, and it defines the structure, rules, and constraints for the elements and attributes within an XML document.
- By using an external DTD, multiple XML documents can share the same set of rules and constraints, leading to more consistency and easier maintenance.
- Following XML document illustrates the use of external DTD.
- Step 1. creation of DTD file
- Step 2 creation of XML document

Merits of DTD

1. DTDs are used to define the structural components of XML document.
2. These are relatively simple and compact
3. DTDs can be defined inline and hence can be embedded directly in the XML document

Demerits of DTD

1. The DTDs are very basic and hence cannot be much specific for complex documents.
2. The language that DTD uses is not an XML document. hence various framework used by XML cannot be supported by DTDs.
3. The DTD cannot define the type of data contained within the XML document hence using DTD we cannot specify whether the element is numeric, or string or of data type.
4. There are some XML processor which do not understand DTDs.
5. The DTDs are not aware of namespace concept.

Schema

- the XML schemas are used to represent the structure of XML document
- The goal or purpose of XML schema is to define the building blocks of an XML document. These can be used as an alternative to XML DTD.
- The XML schema language is called as XML Schema Definition (XSD) language.
- XML schema defines elements, attributes, elements having child elements, order of child elements. It also defines fixed and default values of elements and attributes.
- XML schema allows the developer to use data types.

How to write a simple schema?

Step 1: we will first write a simple xsd file in which the desired structure of the xml document is defined.

I have named it as studentschema.xsd

This file contains the complex type with four child simple type elements

Xml schema (studentschema.xsd)

XML Schema [StudentSchema.xsd]

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="Student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="address" type="xs:string"/>
        <xs:element name="std" type="xs:string"/>
        <xs:element name="marks" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```


Script explanation

The xs is qualifier used to identify the schema elements and types.

The document element of schema is xs:schema. The xs:schema is the root element. It takes attribute xmlns:xs which has the value

<http://www.w3.org/2001/XMLSchema>. this declaration indicates that document should follow the rules of xml schema.

Then comes xs:element which is used to define the xml element . In above case the element student is of complex type who have four child elements : name,address,std and marks.

Step 2: now develop xml document in which the desired values to the xml elements can be given. I have named this file as myschema.xml

XML document (myschema.xml)

XML Document [MySchema.xml]

```
<?xml version="1.0" encoding="UTF-8"?>  
<Student xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:noNamespaceSchemaLocation="StudentSchema.xsd">  
  <name>Anand</name>  
  <address>Pune</address>  
  <std>Second</std>  
  <marks>70 percent</marks>  
</Student>
```

Program explanantion

1. In above xml document, the first line is typical in which the version and encoding is specified
2. There is xmlns:xsi attribute of document which indicated that xml document is an instance of XML schema. Ans it has come from the namespace
<http://www.w3.org/2001/xmlschema-instance>
3. To tie this XML document with the some schema definition we use the attribute xsi:schemaLocation. The value that can be passed to this attribute is the name of xsd file “studentschema.xsd”
- 4 . After this we can define the child elements.

Mozilla Firefox

File Edit View History Bookmarks Tools Help



file:///d:/xml_examples/MySchema.xml



Google

Most Visited Getting Started Latest Headlines

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
- <Student xsi:noNamespaceSchemaLocation="StudentSchema.xsd">  
  <name>Anand</name>  
  <address>Pune</address>  
  <std>Second</std>  
  <marks>70 percent</marks>  
</Student>
```

Data Types -

1. string data type - The string data type can contain characters, line feeds, carriage returns, and tab characters.

➤ The following is an example of a string declaration in a schema:

```
<xs:element name="customer" type="xs:string"/>
```

2. date data type - The date data type is used to specify a date.

➤ The date is specified in the following form "YYYY-MM-DD" where:

YYYY indicates the year

MM indicates the month

DD indicates the day

➤ Note: All components are required!

```
<xs:element name="date_of_birth" type="xs:date"/>
```

3. Time Data Type

➤ The time data type is used to specify a time.

➤ The time is specified in the following form "hh:mm:ss" where:

hh indicates the hour

mm indicates the minute

ss indicates the second

Note: All components are required!

➤ The following is an example of a time declaration in a schema:

```
<xs:element name="start" type="xs:time"/>
```


4. Numeric Data Types – if we want to use numeric value for some element then we can use the data types as either decimal or integer

- Decimal Data Type
- The decimal data type is used to specify a numeric value.
- The following is an example of a decimal declaration in a schema:
- `<xs:element name="price" type="xs:decimal"/>`

5. Integer Data Type

- The integer data type is used to specify a numeric value without a fractional component.
- The following is an example of an integer declaration in a schema:
- `<xs:element name="price" type="xs:integer"/>`

- 6 Boolean Data Type - The boolean data type is used to specify a true or false value.
- The following is an example of a boolean declaration in a schema:
 - `<xs:attribute name="disabled" type="xs:boolean"/>`
 - Note: Legal values for boolean are true, false, 1 (which indicates true), and 0 (which indicates false).

Advantages and disadvantages of schema

- Advantages of schema -

- 1 The schema are more specific
- 2 The schema provide the support for data type.
- 3 The schema is aware of namespace xml schema is the W3C recommendation. Hence it is supported by various XML validator and XML processors
4. The XML schema is written in XML itself and has a large number of built in and derived types
5. The XML schema is the W3C recommendation. Hence it is supported by various XML validator and XML processors

disadvantages of schema

1. The XML schema is complex to design and hard to learn.
2. The XML document cannot be if the corresponding schema file is absent.
3. Maintaining the schema for large and complex operations sometimes slows down the processing XML document

Advantages of schema over DTD

1. Both the schemas and DTDs are useful for defining structural components of XML. But the DTDs are basic and cannot be much specific for complex operations. On the other hand schemas are more specific.
2. The schemas provide support for defining the type of data. The DTDs do not have this ability. Hence content definition is possible using schema.
3. The schemas are namespace aware and DTDs are not.
4. The XML schema is written in XML itself and has a large number of built in and derived types.
5. The schema is the W3C recommendation. Hence it is supported by various XML validator and XML processors but there are some XML processors which do not support DTD.
6. Large number of web applications can be built using XML schema. On the other hand relatively simple and compact operations can be built using DTD.

DOM based XML processing

➤ The DOM defines a standard for accessing and manipulating documents:

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."

➤ The HTML DOM defines a standard way for accessing and manipulating HTML documents. It presents an HTML document as a tree-structure.

➤ Thus XML DOM is for –

➤ Loading the xml document

➤ Accessing the elements of XML document

➤ Deleting the elements of XML document

➤ Changing the elements of xml document

Transforming XML document

- The XSLT stands for XSL transforming and XSL stands for extensible stylesheet language.
 - An XSLT is a WWW3C recommendation
 - The XSLT is used for defining the XML document transformations and presentation. Hence XSL decides how an XML document should look on the web browser.
1. XSL consists of three parts – 1. XSLT is a language for transforming XML document.
 2. XPath is a language for navigating in XML documents. In other words we can reach to any node of xml document using XPath.
 3. XSL-FO is a language for formatting XML documents for displaying it in desired manner

Transforming XML into XSLT

- Using XSLT we can decide the way by which we want the element to get displayed.
- XSLT uses XPATH to find information in an XML document. Xpath is used to navigate through elements and attributes in XML document. we can sort these elements, hide or display particular element and can apply some decision making logic on these elements.
- XSLT processors take two input documents one is XML document and another is XSLT document. The XSLT document is nothing but a program and XML document is nothing but the input data., thus this program works on the XML input data.
- This newly produced document is provided as input to XSLT processor which in turn produces another document called XSL document
- The XSL document is used along with application so that particular application can be displayed on the web browser in some desired manner.

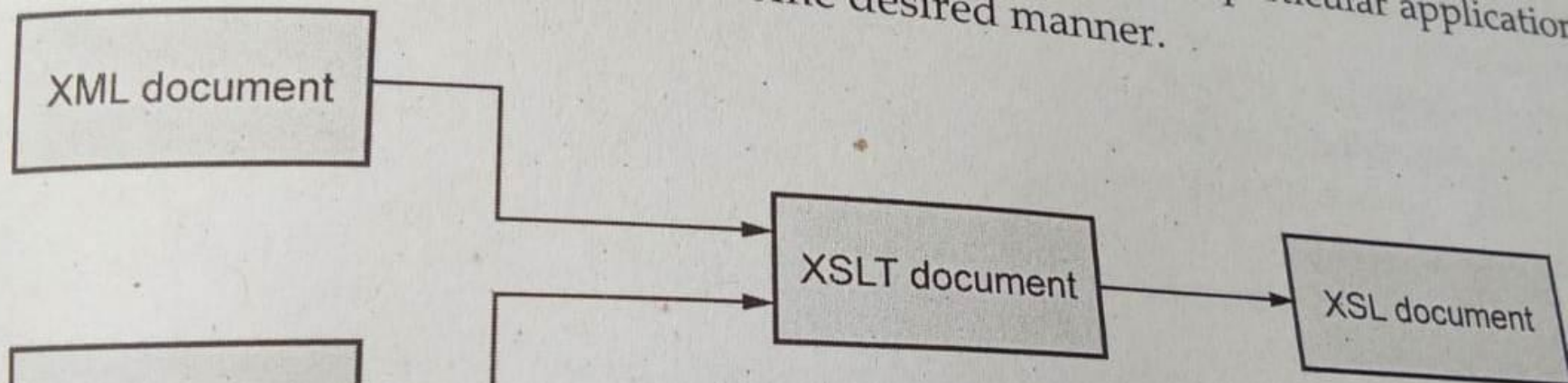


Fig. 3.22.1 Processing of XSLT document

Difference between XML and XSLT

Sr.No.	XML	XSLT
1.	XML stands for extensible markup language.	XSLT stands for extensible stylesheet language transformations.
2.	XML is used to store data in structured format.	XSLT is used for transforming and formatting the XML file.
3.	XML does not perform transformations.	XSLT can transform one XML format into another XML format or into HTML or into plain text.
4.	XML contains the user defined tags.	XSLT contains special XSL tags and programming constructs.
5.	XML uses the user defined raw data for storage.	XSLT uses XPath for transformation and formatting of XML file.

AJAX

AJAX is a Asynchronous Java Script and XML

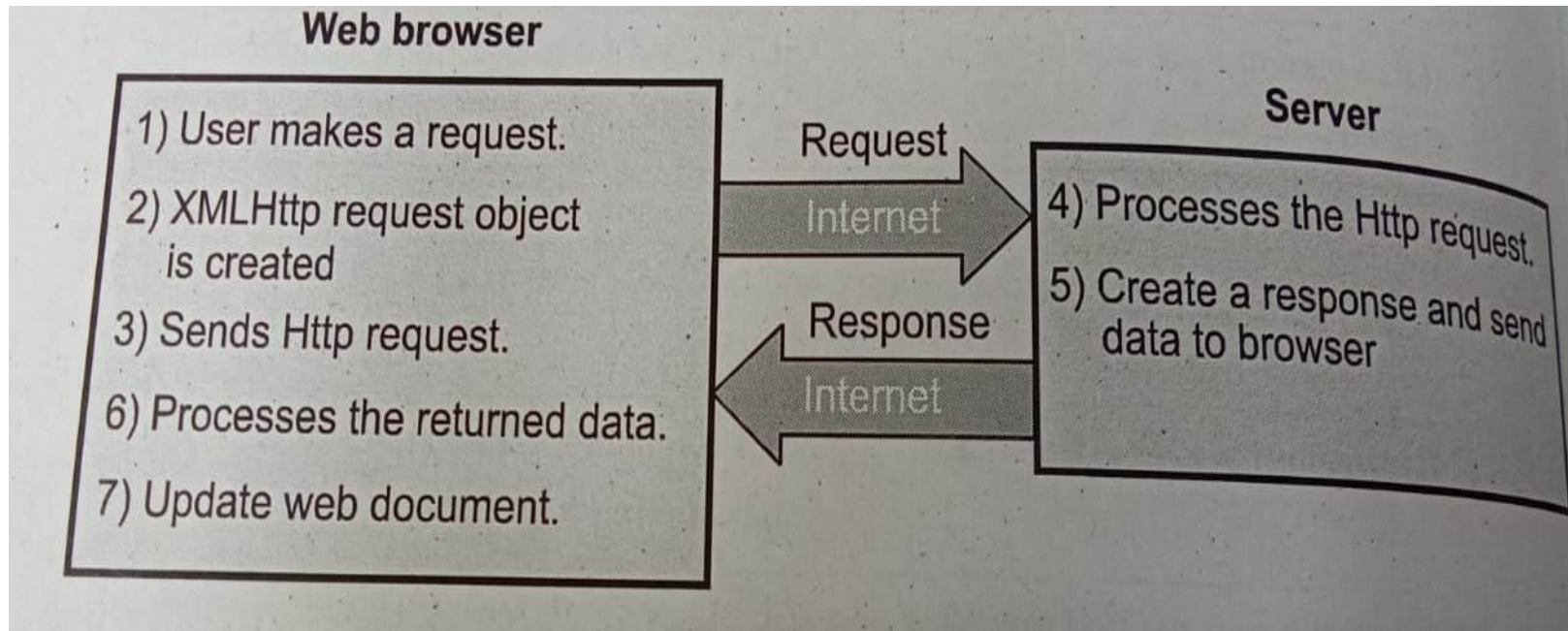
Here,

- Asynchronous means, the execution of script does not disturb the user's work.
- JavaScript because, it makes use of JavaScripting to do the actual work.
- XML because along with JavaScripting to do the actual work
- it is not a new programming language but it is a kind of web document which adopts certain standards
- Ajax allows the developer to exchange the data with the server and updates the part of web document without reloading the web page

Working of AJAX

- When user makes a request , the browser creates a object for the HttpRequest and a request is made to the server over an internet.
- The processes this request and stands the required data to the browser.
- At the browser side the returned data is processed using JavaScript and the web document gets uploaded accordingly by sending the appropriate response.

Working of AJAX



Similarities and Differences between Traditional Web Application and AJAX Based web Application Architecture

Similarities and Differences

Sr.No.	Traditional Web Application Architecture	Ajax Based Web Application Architecture
1.	The web applications architectures are based on HTTP request response protocol.	
2.	At the client side the browser client has only one component and that is – user interface.	At the client side the browser client has user interface and AJAX Engine due to which the client gets quick response from the server.
3.	The architecture is based on client server communication.	
4.	It makes use of synchronous communication. The client has to wait to get the response from the server then only client can make the request to the server. It is start-stop-start-stop kind of communication.	By adding a new layer of AJAX Engine at the client side, it eliminates the start-stop-start-stop nature of communication and makes the communication asynchronous.
5.	With traditional web application architecture user cannot get rich user interface experience.	With AJAX architecture user gets rich user interface experience.
6.	It is less responsive.	It is more responsive.

7. Building web application is simple.

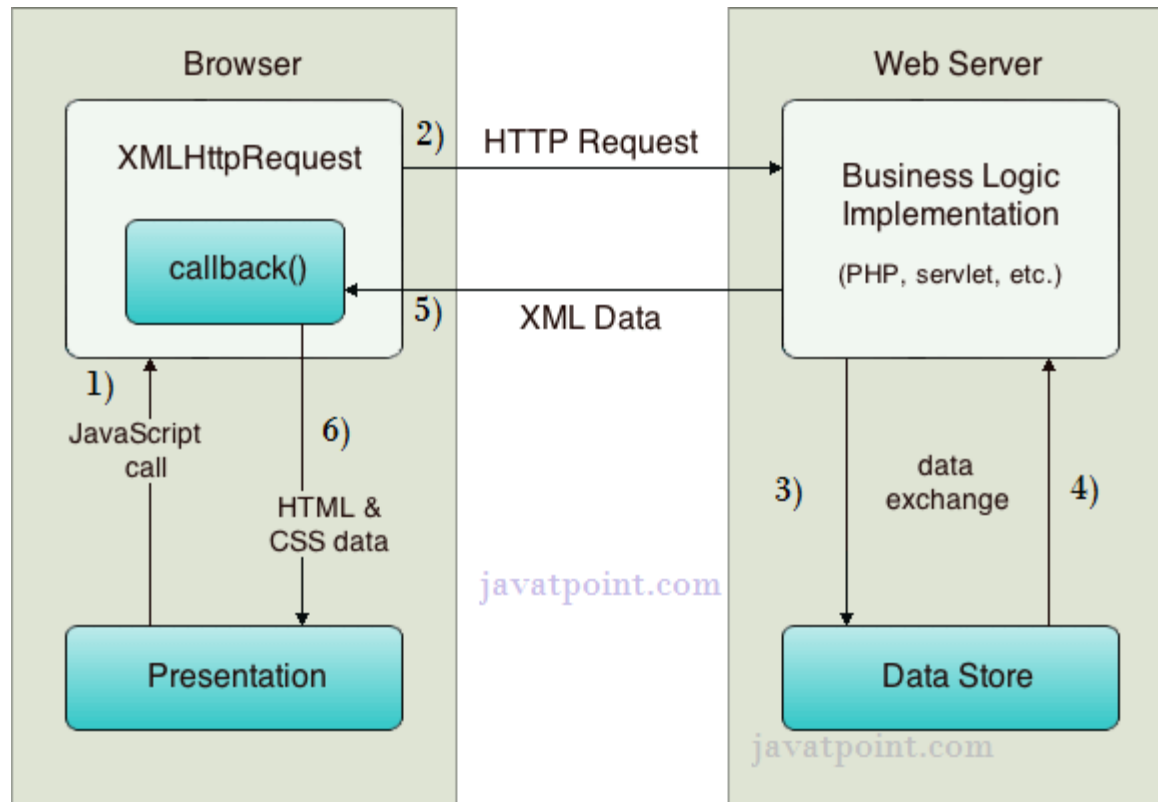
The development time is more while designing the Ajax based web application.

8. Limited use of JavaScript, CSS and XML technologies. In fact - Most user actions in the interface trigger an HTTP request back to a web server.

Extensive use of JavaScript, CSS and XML -

- i) standards-based presentation using XHTML and CSS;
- ii) Dynamic display and interaction using the Document Object Model(DOM);
- iii) Data interchange and manipulation using XML and XSLT;
- iv) Asynchronous data retrieval using XMLHttpRequest;
- v) JavaScript binding everything together

How AJAX works?



3. Server interacts with the database using JSP, PHP, Servlet, ASP.net etc.
4. Data is retrieved.
5. Server sends XML data or JSON data to the XMLHttpRequest callback function.
6. HTML and CSS data is displayed on the browser.

How AJAX works?

Merits of AJAX

- Response time is faster, hence increases performance and speed
- XMLHttpRequest object call as a asynchronous HTTP request to the server for transferring data both side. Its used for making requests to the non – Ajax pages
- It is used in form validation
- It performs fetching of data from database and storing of data into database without reloading page

Demerits of AJAX

- It has browsing compatibility issues
- It is impossible to bookmark AJAX updated page contents
- Search engine would not crawl AJAX generated content.hence search engines like Google cannot index AJAX pages.

Coding AJAX script

```
<html>
<head>
<script type = "" text/javascript">
Function Myfun()
{
    if(window.XMLHttpRequest)
    {
        req = new XMLHttpRequest();
    }
    else
    {
        req = new ActiveXObject("Microsoft.req");
    }
    Req.onreadystatechange = function()
    {
        if(req.readyState == 4&& req.staatus == 200)
        {
            document.getElementById("myID").innerHTML = req.responseText;
        }
    }
    req.open("GET","newdata.txt",true);
    req.send();
}
```

The diagram illustrates the execution flow of the AJAX script with four numbered annotations in blue ovals:

- 2**: A bracket on the right side of the `if(window.XMLHttpRequest)` block, indicating the execution of the `XMLHttpRequest` object creation.
- 3**: A bracket on the right side of the `if(req.readyState == 4&& req.staatus == 200)` block, indicating the execution of the `document.getElementById` call to update the HTML content.
- 4**: A bracket on the right side of the `req.open("GET","newdata.txt",true);` line, indicating the execution of the `open` method.
- 5**: A bracket on the right side of the `req.send();` line, indicating the execution of the `send` method.

```
<body>  
< div id = "myID"> This text can be changed </div>  
<button type = "button" onclick = "MyFun()">Change</button>  
</body>  
</html>
```

Script explanation

1. On button click function Myfun is invoked . Thus client triggers the event.
2. XMLHttpRequest object is used to exchange data with a server. This object allows the user to change/update the parts of web page without reloading it fully.
The modern web browser such as Firefox, chrome have built in XMLHttpRequest but old web browsers make use of XMLHttpRequest
3. When a request to a server is sent , then onreadystatechange event is triggered.
The readystate property holds the status of XMLHttpRequest. The readystate=4 means request is finished and response is ready. The status =200 means “ok”
The DOM is modified and response is modified using the statement
`document.getElementById(“myID”).innerHTML = req.responseText;`
4. The request can be sent to the server by using two functions open() and send()

GET or POST method

True : means asynchronous
False : means synchronous

Location of the file on server

Asynchronous communication allows fast processing of the data.

The newdata.txt file contains some updating text using which our web page can be updated

5. The send() method sends the request to the server.

